AUTOM-07: CI/CD Integration

> **Series:** AUTOM — Dynatrace Automation | **Notebook:** 7 of 8 | **Created:** January 2026 | **Last Updated:** 05/11/2026

CI/CD integration brings software development practices to Dynatrace configuration management. By storing configs in Git and deploying via pipelines, teams gain version control, review processes, and automated deployments.

Table of Contents

Prerequisites

Before starting this notebook, ensure you have:

Requirement	Description
CI/CD Platform	GitHub Actions, GitLab CI, or Jenkins
Monaco or Terraform	One of the config-as-code tools
Git Repository	For storing configurations
Authentication	API Token + Platform Token for full coverage (see AUTOM-04)
HashiCorp Vault	*Optional* — for runtime credential retrieval instead of static CI/CD secrets
OPA / Conftest	*Optional* — for policy-as-code gates in pipelines

Token Types Reminder

As of Dynatrace Terraform provider **v1.88.0**, synthetic monitors and SLOs require a classic **API Token** (`dt0c01`). For full resource coverage in pipelines, use **Platform Token + API Token** together. See AUTOM-04: Terraform Provider for details.

Learning Objectives

By the end of this notebook, you will:

- Understand GitOps principles for Dynatrace
- Know how to set up CI/CD pipelines for config deployment
- Be able to implement pull request workflows with plan-as-PR-comment
- Handle multi-environment deployments with combined auth
- Integrate HashiCorp Vault for runtime credential retrieval
- Implement policy-as-code gates with OPA/Conftest and Sentinel
- Set up automated drift detection workflows
- Create reusable GitHub Actions workflows for multi-team organizations

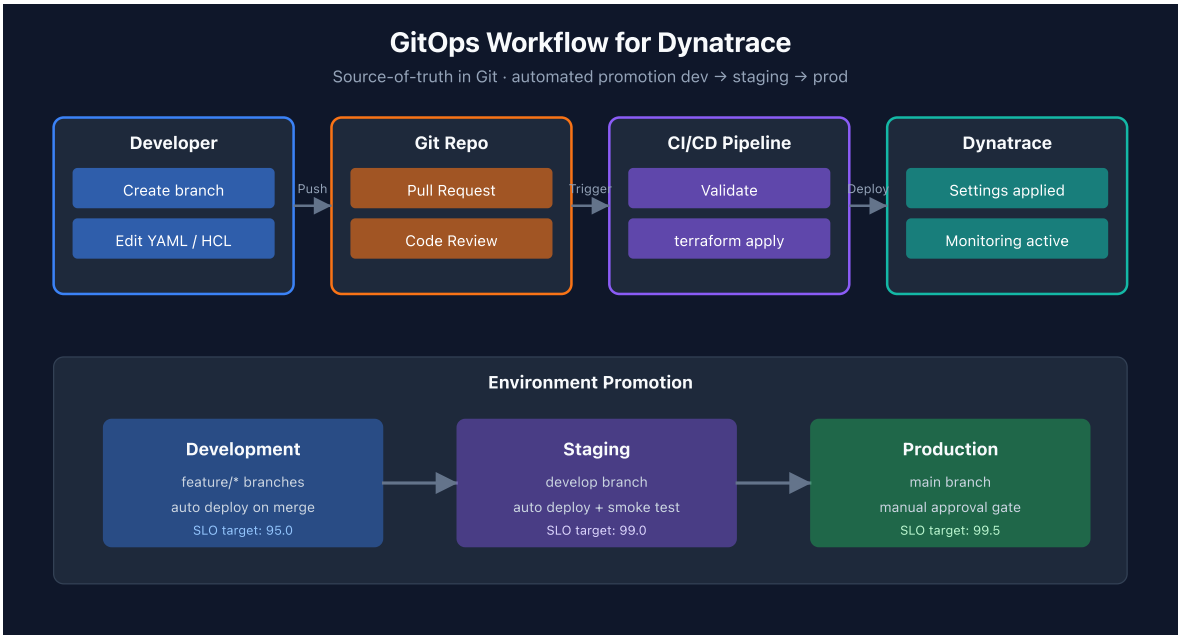
---

1. Introduction

Why CI/CD for Dynatrace?

Benefit	Description
Version Control	All changes tracked in Git history
Code Review	Pull requests for config changes
Automation	No manual deployments
Rollback	Revert to any previous state
Audit Trail	Who changed what, when

GitOps Workflow



2. GitOps Fundamentals

Repository Structure

```
dynatrace-config/
├── .github/
│   └── workflows/
│       ├── validate.yml
│       └── deploy.yml
├── manifest.yaml          # Monaco manifest
├── environments/
│   ├── dev.yaml
│   ├── staging.yaml
│   └── production.yaml
├── projects/
│   ├── alerting/
│   ├── management-zones/
│   └── synthetic/
└── README.md
```

Branch Strategy

Branch	Purpose	Deploys To
`main`	Production configs	Production
`develop`	Integration branch	Staging
`feature/*`	New features	Dev (optional)

Environment Promotion

```
feature/* → develop → main
    ↓          ↓        ↓
   Dev     Staging   Production
```

3. GitHub Actions

Validation Workflow

.github/workflows/validate.yml:

```
name: Validate Dynatrace Config

on:
  pull_request:
    branches: [main, develop]
    paths:
      - 'projects/**'
      - 'manifest.yaml'

jobs:
  validate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v6

      - name: Install Monaco
        run: |
          curl -L https://github.com/Dynatrace/dynatrace-configuration-as-
code/releases/latest/download/monaco-linux-amd64 -o monaco
          chmod +x monaco
          sudo mv monaco /usr/local/bin/

      - name: Validate Configuration
        run: monaco validate manifest.yaml

      - name: Dry Run Deploy
        env:
          DT_DEV_URL: ${{ secrets.DT_DEV_URL }}
          DT_DEV_TOKEN: ${{ secrets.DT_DEV_TOKEN }}
        run: |
          monaco deploy manifest.yaml --environment development --dry-run
```

Deployment Workflow

.github/workflows/deploy.yml:

```yaml
name: Deploy Dynatrace Config

on:
  push:
    branches: [main]
    paths:
      - 'projects/**'
      - 'manifest.yaml'

jobs:
  deploy-staging:
    runs-on: ubuntu-latest
    environment: staging
    steps:
      - uses: actions/checkout@v6

      - name: Install Monaco
        run: |
          curl -L https://github.com/Dynatrace/dynatrace-configuration-as-
code/releases/latest/download/monaco-linux-amd64 -o monaco
          chmod +x monaco
          sudo mv monaco /usr/local/bin/

      - name: Deploy to Staging
        env:
          DT_STAGING_URL: ${{ secrets.DT_STAGING_URL }}
          DT_STAGING_TOKEN: ${{ secrets.DT_STAGING_TOKEN }}
        run: |
          monaco deploy manifest.yaml --environment staging

  deploy-production:
    needs: deploy-staging
    runs-on: ubuntu-latest
    environment: production
    steps:
      - uses: actions/checkout@v6

      - name: Install Monaco
        run: |
          curl -L https://github.com/Dynatrace/dynatrace-configuration-as-
code/releases/latest/download/monaco-linux-amd64 -o monaco
          chmod +x monaco
          sudo mv monaco /usr/local/bin/

      - name: Deploy to Production
        env:
          DT_PROD_URL: ${{ secrets.DT_PROD_URL }}
          DT_PROD_TOKEN: ${{ secrets.DT_PROD_TOKEN }}
        run: |
          monaco deploy manifest.yaml --environment production
```

Terraform Workflow with Combined Auth

For Terraform-based deployments, use **Platform Token + API Token** together for full resource coverage. The provider automatically routes each resource to the correct token.

```yaml
name: Terraform Dynatrace

on:
  pull_request:
    branches: [main]
  push:
    branches: [main]

jobs:
  terraform:
    runs-on: ubuntu-latest
    permissions:
      contents: read
      pull-requests: write
    steps:
      - uses: actions/checkout@v6

      - name: Setup Terraform
        uses: hashicorp/setup-terraform@v4
        with:
          terraform_version: latest

      - name: Terraform Init
        run: terraform init

      - name: Terraform Plan
        id: plan
        if: github.event_name == 'pull_request'
        run: terraform plan -no-color -out=tfplan
        env:
          DYNATRACE_ENV_URL: ${{ secrets.DT_ENV_URL }}
          DYNATRACE_PLATFORM_TOKEN: ${{ secrets.DT_PLATFORM_TOKEN }}
          DYNATRACE_API_TOKEN: ${{ secrets.DT_API_TOKEN }}
          DYNATRACE_HTTP_OAUTH_PREFERENCE: "true"

      - name: Post Plan to PR
        if: github.event_name == 'pull_request'
        uses: borchero/terraform-plan-comment@v2
        with:
          token: ${{ github.token }}
          planfile: tfplan

      - name: Terraform Apply
        if: github.ref == 'refs/heads/main' &amp;&amp; github.event_name == 'push'
        run: terraform apply -auto-approve
        env:
          DYNATRACE_ENV_URL: ${{ secrets.DT_ENV_URL }}
          DYNATRACE_PLATFORM_TOKEN: ${{ secrets.DT_PLATFORM_TOKEN }}
          DYNATRACE_API_TOKEN: ${{ secrets.DT_API_TOKEN }}
          DYNATRACE_HTTP_OAUTH_PREFERENCE: "true"
```

Secret	Token Type	Covers
`DT_PLATFORM_TOKEN`	`dt0s16.xxxx`	Settings 2.0 + Gen3 Platform (workflows, documents, segments)
`DT_API_TOKEN`	`dt0c01.xxxx`	Synthetic monitors, SLOs (v1.88.0 requirement)
`DT_ENV_URL`	—	Tenant URL

> **Why combined auth?** A single API Token cannot manage Gen3 resources. A single Platform Token cannot manage synthetics/SLOs (removed in v1.88.0). Using both together gives the pipeline full coverage.

Vault Integration for Runtime Credentials

For enterprise environments with **short-lived tokens** (e.g., 24-hour expiration), retrieve credentials at runtime from **HashiCorp Vault** instead of storing them as static CI/CD secrets.

```
name: Terraform with Vault Credentials

on:
  pull_request:
    branches: [main]
  push:
    branches: [main]

jobs:
  terraform:
    runs-on: ubuntu-latest
    permissions:
      id-token: write       # Required for OIDC → Vault JWT auth
      contents: read
      pull-requests: write
    steps:
      - uses: actions/checkout@v6

      - name: Retrieve Dynatrace credentials from Vault
        uses: hashicorp/vault-action@v3
        with:
          url: ${{ secrets.VAULT_ADDR }}
          method: jwt
          role: dynatrace-deployer
          secrets: |
            secret/data/dynatrace/prod platform_token | DYNATRACE_PLATFORM_TOKEN ;
            secret/data/dynatrace/prod api_token | DYNATRACE_API_TOKEN ;
            secret/data/dynatrace/prod env_url | DYNATRACE_ENV_URL

      - name: Setup Terraform
        uses: hashicorp/setup-terraform@v4

      - name: Terraform Init
        run: terraform init

      - name: Terraform Plan
        run: terraform plan -no-color -out=tfplan
        env:
          DYNATRACE_HTTP_OAUTH_PREFERENCE: "true"

      - name: Terraform Apply
        if: github.ref == 'refs/heads/main' &amp;amp;&amp;amp; github.event_name == 'push'
        run: terraform apply -auto-approve
        env:
          DYNATRACE_HTTP_OAUTH_PREFERENCE: "true"
```

Vault path structure per team per tenant:

```
secret/
└── dynatrace/
    ├── dev/
    │   ├── platform_token    # dt0s16 for dev tenant
    │   ├── api_token         # dt0c01 for dev tenant
    │   └── env_url           # https://{dev-tenant}.live.dynatrace.com
    ├── prod/
    │   ├── platform_token
    │   ├── api_token
    │   └── env_url
    └── teams/
        ├── payments/         # Team-specific OAuth clients
        │   ├── client_id
        │   └── client_secret
        └── platform/
            ├── client_id
            └── client_secret
```

> **Why Vault?** Static GitHub Secrets don't expire. If your organization uses 24-hour token rotation or needs audit trails for credential access, Vault provides runtime retrieval with automatic expiration and access logging.

Policy-as-Code Gates (OPA / Sentinel)

Add governance checks to your pipeline using **OPA/Conftest** (open-source) or **Sentinel** (Terraform Enterprise/Cloud).

OPA/Conftest — Resource Type Allowlist

Restrict which Dynatrace resource types a team can create:

policy/allowed_resources.rego:

```
package main

# Only allow these Dynatrace resource types
allowed_resources := {
  "dynatrace_alerting",
  "dynatrace_autotag_v2",
  "dynatrace_management_zone_v2",
  "dynatrace_maintenance"
}

deny[msg] {
  resource := input.planned_values.root_module.resources[_]
  startswith(resource.type, "dynatrace_")
  not allowed_resources[resource.type]
  msg := sprintf("Resource type '%s' is not in the allowlist", [resource.type])
}
```

OPA/Conftest — Mandatory Team Tagging

Ensure all resources include team ownership metadata:

policy/mandatory_tags.rego:

```
package main

deny[msg] {
  resource := input.planned_values.root_module.resources[_]
  resource.type == "dynatrace_autotag_v2"
  not resource.values.name
  msg := "Auto-tag resources must have a name"
}

deny[msg] {
  resource := input.planned_values.root_module.resources[_]
  startswith(resource.type, "dynatrace_")
  not has_team_ownership(resource)
  msg := sprintf("Resource '%s' must include team ownership metadata",
[resource.address])
}

has_team_ownership(resource) {
  contains(resource.values.name, resource.values.team_name)
}
```

Pipeline Integration

```
# Add after terraform plan step
- name: Install Conftest
  run: |
    wget -q https://github.com/open-policy-
agent/conftest/releases/latest/download/conftest_Linux_x86_64.tar.gz
    tar xzf conftest_Linux_x86_64.tar.gz
    sudo mv conftest /usr/local/bin/

- name: Run Policy Checks
  run: |
    terraform show -json tfplan &gt; tfplan.json
    conftest test tfplan.json --policy policy/
```

Sentinel (Terraform Enterprise / Cloud)

For teams using **TFE** (now HCP Terraform), Sentinel policies enforce governance at the workspace level:

```
# sentinel/restrict_resource_types.sentinel
import "tfplan/v2" as tfplan

allowed_types = [
  "dynatrace_alerting",
  "dynatrace_autotag_v2",
  "dynatrace_management_zone_v2",
]

main = rule {
  all tfplan.resource_changes as _, rc {
    rc.type in allowed_types or not (rc.type matches "dynatrace_.*")
  }
}
```

> **Important — Sentinel Limitations:** Sentinel evaluates Terraform **plans** — it does not grant or restrict Dynatrace API access at runtime. It cannot reduce the permissions of a token that has broad scope. If a pipeline token can write all Synthetic monitors, Sentinel cannot narrow that. Use **Dynatrace IAM policies** (see **AUTOM-04: IAM Policy Management**) for platform-level access control, and Sentinel for pipeline-level governance. Sentinel only runs in **HCP Terraform** — it is not available in open-source Terraform, GitHub Actions, or plain CI runners. For non-HCP environments, use **OPA/Conftest** as shown above.

Drift Detection Workflow

Schedule automatic drift detection to catch manual UI changes ("ClickOps") that bypass your Terraform pipeline:

```yaml
name: Drift Detection

on:
  schedule:
    - cron: '0 6 * * 1-5'   # Every weekday at 6am UTC
  workflow_dispatch: {}       # Allow manual trigger

jobs:
  detect-drift:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v6

      - name: Setup Terraform
        uses: hashicorp/setup-terraform@v4

      - name: Terraform Init
        run: terraform init

      - name: Check for Drift
        id: drift
        run: |
          set +e
          terraform plan -detailed-exitcode -no-color 2&gt;&amp;1 | tee drift-
output.txt
          echo "exitcode=$?" &gt;&gt; "$GITHUB_OUTPUT"
        env:
          DYNATRACE_ENV_URL: ${{ secrets.DT_ENV_URL }}
          DYNATRACE_PLATFORM_TOKEN: ${{ secrets.DT_PLATFORM_TOKEN }}
          DYNATRACE_API_TOKEN: ${{ secrets.DT_API_TOKEN }}
          DYNATRACE_HTTP_OAUTH_PREFERENCE: "true"

      - name: Create Issue on Drift
        if: steps.drift.outputs.exitcode == '2'
        uses: actions/github-script@v7
        with:
          script: |
            const fs = require('fs');
            const drift = fs.readFileSync('drift-output.txt', 'utf8').slice(-3000);
            await github.rest.issues.create({
              owner: context.repo.owner,
              repo: context.repo.repo,
              title: '⚠️ Dynatrace configuration drift detected',
              body: `Scheduled drift detection found changes not in Terraform
state.\n\n\`\`\`\n${drift}\n\`\`\`\n\nSomeone may have made manual changes in the
Dynatrace UI.`,
              labels: ['drift', 'dynatrace']
            });
```

> **Exit codes:** `terraform plan -detailed-exitcode` returns `0` = no changes, `1` = error, `2` = drift detected. This lets your pipeline take different actions based on the result.

Reusable GitHub Actions Workflows

For organizations managing multiple Dynatrace tenants or LOB repositories, create **reusable workflows** that any repo can call:

.github/workflows/terraform-dynatrace.yml (in a shared workflow repo):

```yaml
name: Terraform Dynatrace (Reusable)

on:
  workflow_call:
    inputs:
      working_directory:
        required: false
        type: string
        default: '.'
      terraform_version:
        required: false
        type: string
        default: '1.6.0'
    secrets:
      DT_ENV_URL:
        required: true
      DT_PLATFORM_TOKEN:
        required: true
      DT_API_TOKEN:
        required: true

jobs:
  plan:
    runs-on: ubuntu-latest
    permissions:
      contents: read
      pull-requests: write
    defaults:
      run:
        working-directory: ${{ inputs.working_directory }}
    steps:
      - uses: actions/checkout@v6

      - name: Setup Terraform
        uses: hashicorp/setup-terraform@v4
        with:
          terraform_version: ${{ inputs.terraform_version }}

      - name: Terraform Init
        run: terraform init

      - name: Terraform Plan
        run: terraform plan -no-color -out=tfplan
        env:
          DYNATRACE_ENV_URL: ${{ secrets.DT_ENV_URL }}
          DYNATRACE_PLATFORM_TOKEN: ${{ secrets.DT_PLATFORM_TOKEN }}
          DYNATRACE_API_TOKEN: ${{ secrets.DT_API_TOKEN }}
          DYNATRACE_HTTP_OAUTH_PREFERENCE: "true"

      - name: Post Plan to PR
        if: github.event_name == 'pull_request'
```

```
        uses: borchero/terraform-plan-comment@v2
        with:
          token: ${{ github.token }}
          planfile: tfplan

  apply:
    needs: plan
    if: github.ref == 'refs/heads/main' &amp;&amp; github.event_name == 'push'
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: ${{ inputs.working_directory }}
    steps:
      - uses: actions/checkout@v6

      - name: Setup Terraform
        uses: hashicorp/setup-terraform@v4
        with:
          terraform_version: ${{ inputs.terraform_version }}

      - name: Terraform Init
        run: terraform init

      - name: Terraform Apply
        run: terraform apply -auto-approve
        env:
          DYNATRACE_ENV_URL: ${{ secrets.DT_ENV_URL }}
          DYNATRACE_PLATFORM_TOKEN: ${{ secrets.DT_PLATFORM_TOKEN }}
          DYNATRACE_API_TOKEN: ${{ secrets.DT_API_TOKEN }}
          DYNATRACE_HTTP_OAUTH_PREFERENCE: "true"
```

Calling the reusable workflow from a LOB repo:

```
# .github/workflows/deploy.yml (in dt-lob-payments repo)
name: Deploy Dynatrace Config

on:
  pull_request:
    branches: [main]
  push:
    branches: [main]

jobs:
  terraform:
    uses: my-org/dt-templates/.github/workflows/terraform-dynatrace.yml@main
    with:
      working_directory: terraform/
    secrets:
      DT_ENV_URL: ${{ secrets.DT_ENV_URL }}
      DT_PLATFORM_TOKEN: ${{ secrets.DT_PLATFORM_TOKEN }}
      DT_API_TOKEN: ${{ secrets.DT_API_TOKEN }}
```

> **Multi-tenant pattern:** For organizations with 5+ tenants, the reusable workflow is called once per environment with different secrets, providing a consistent deployment experience across all tenants.

---

4. GitLab CI/CD

Pipeline Configuration

.gitlab-ci.yml:

```
stages:
  - validate
  - deploy-staging
  - deploy-production

.monaco-setup: &amp;monaco-setup
  before_script:
    - curl -L https://github.com/Dynatrace/dynatrace-configuration-as-
code/releases/latest/download/monaco-linux-amd64 -o monaco
    - chmod +x monaco
    - mv monaco /usr/local/bin/

validate:
  stage: validate
  &lt;&lt;: *monaco-setup
  script:
    - monaco validate manifest.yaml
    - monaco deploy manifest.yaml --environment development --dry-run
  rules:
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"

deploy-staging:
  stage: deploy-staging
  &lt;&lt;: *monaco-setup
  script:
    - monaco deploy manifest.yaml --environment staging
  environment:
    name: staging
  rules:
    - if: $CI_COMMIT_BRANCH == "main"

deploy-production:
  stage: deploy-production
  &lt;&lt;: *monaco-setup
  script:
    - monaco deploy manifest.yaml --environment production
  environment:
    name: production
  rules:
    - if: $CI_COMMIT_BRANCH == "main"
      when: manual
```

Variables Configuration

In GitLab: **Settings → CI/CD → Variables**

Variable	Protected	Masked
`DT_DEV_URL`	No	No

Variable	Protected	Masked
`DT_DEV_TOKEN`	No	Yes
`DT_STAGING_URL`	No	No
`DT_STAGING_TOKEN`	Yes	Yes
`DT_PROD_URL`	Yes	No
`DT_PROD_TOKEN`	Yes	Yes

---

5. Bitbucket Pipelines

Bitbucket Cloud (bitbucket.org) ships **Bitbucket Pipelines** as the integrated CI/CD service. The pipeline definition lives in a `bitbucket-pipelines.yml` file at the repository root. Variables (including secrets) live at three scopes: workspace, repository, and deployment-environment.

> **Bitbucket Data Center / Server** uses a different mechanism (Bitbucket Pipelines is a Cloud-only feature). For self-hosted Bitbucket, integrate via Jenkins, GitLab CI, or another CI/CD tool — the patterns below assume Bitbucket Cloud.
>
> ### Provider landscape note (mid-2026)
>
> The Terraform-provider landscape for Bitbucket Cloud is in active transition:
>
> - **`DrFaust92/bitbucket`** — the long-standing community provider — entered **maintenance mode** in March 2026. The maintainer is no longer using Bitbucket and has limited bandwidth for substantive changes.
> - **Bitbucket App Passwords are being retired by Atlassian** in favor of API tokens (see App passwords (Atlassian) and API tokens (Atlassian)). Check the current Atlassian deprecation notice for the exact cutover date — once it lands, `DrFaust92` users need to swap their `BITBUCKET_PASSWORD` from an App Password to an API token (workaround community-validated: use the Atlassian-account email as `BITBUCKET_USERNAME` + an API token as `BITBUCKET_PASSWORD`, with `-parallelism=2` to avoid throttling).
> - **`FabianSchurig/bitbucket`** — a newer, actively-maintained provider — is the going-forward recommendation. It uses a "generic resources" architecture (resources expose Bitbucket Cloud API endpoints directly via typed core fields plus `request_body` for full API flexibility) and a simplified API-token-only auth. As of May

> 2026 it is at v0.15.x — **pre-1.0, with schema flux possible** — so the examples below should be verified against the provider docs at use time.
>
> The examples in this section use `FabianSchurig/bitbucket`. For teams with existing `DrFaust92` code, the migration guide covers resource-name mapping, auth changes, and path-parameter renames (`owner` → `workspace`, `repository` → `repo_slug`).

Pipeline — Monaco Deploy

bitbucket-pipelines.yml at repo root:

```
image: atlassian/default-image:4

definitions:
  steps:
    - step: &amp;install-monaco
        name: Install Monaco
        script:
          - curl -L https://github.com/Dynatrace/dynatrace-configuration-as-
code/releases/latest/download/monaco-linux-amd64 -o monaco
          - chmod +x monaco
          - mv monaco /usr/local/bin/

pipelines:
  pull-requests:
    '**':
      - step:
          &lt;&lt;: *install-monaco
          name: Validate Monaco config
          script:
            - monaco validate manifest.yaml
            - monaco deploy manifest.yaml --environment development --dry-run

  branches:
    main:
      - step:
          &lt;&lt;: *install-monaco
          name: Deploy to staging
          deployment: staging
          script:
            - monaco deploy manifest.yaml --environment staging

      - step:
          &lt;&lt;: *install-monaco
          name: Deploy to production
          deployment: production
          trigger: manual
          script:
            - monaco deploy manifest.yaml --environment production
```

Each step declared with `deployment: <env>` inherits that environment's deployment variables. The `trigger: manual` on the production step gates the deploy behind a manual confirmation in the Bitbucket UI.

Pipeline — Terraform with Combined Auth

Mirrors the GitHub Actions Terraform pattern (§3.1) — Platform Token + API Token together for full Dynatrace resource coverage:

```
image: hashicorp/terraform:1.6

pipelines:
  pull-requests:
    '**':
      - step:
          name: Terraform Plan
          deployment: staging
          script:
            - terraform init
            - terraform plan -no-color -out=tfplan
            - terraform show -no-color tfplan &gt; plan.txt
          artifacts:
            - plan.txt
            - tfplan

  branches:
    main:
      - step:
          name: Terraform Apply (staging)
          deployment: staging
          script:
            - terraform init
            - terraform apply -auto-approve

      - step:
          name: Terraform Apply (production)
          deployment: production
          trigger: manual
          script:
            - terraform init
            - terraform apply -auto-approve
```

The pipeline reads `DYNATRACE_ENV_URL`, `DYNATRACE_PLATFORM_TOKEN`, `DYNATRACE_API_TOKEN`, and `DYNATRACE_HTTP_OAUTH_PREFERENCE` from the deployment environment's variables — see Variables and Secrets below.

Terraform Bitbucket Provider — FabianSchurig (recommended for new code)

The **`FabianSchurig/bitbucket`** provider manages Bitbucket Cloud resources via the Cloud API. Architecture quirks worth knowing before reading the example:

- **Typed core fields + `request_body` escape hatch.** Each resource has a few typed arguments for the most-common fields (`workspace`, `repo_slug`, `key`, `secured`, etc.) plus a `request_body` JSON string for full API payload flexibility. When a field you need is not surfaced as a typed argument, you set it via `request_body = jsonencode({...})`.

- **Boolean fields are strings.** `secured = "true"`, `enabled = "true"` — typed as string, not bool. The generic-resource architecture preserves API string representations.
- **Auth is API-token only.** Set the `token` argument on the provider (or `BITBUCKET_TOKEN` env var). No App Password, no OAuth.

Key resources used in this section:

Resource	What it manages
`bitbucket_repos`	The repo itself (CRUD); core fields + `request_body` for less-common settings
`bitbucket_pipeline_config`	Enables/disables Bitbucket Pipelines on the repo (`enabled = "true"`)
`bitbucket_repo_settings`	Repository settings split out of the legacy repo resource
`bitbucket_deployments`	A deployment environment (named — staging / production / etc.)
`bitbucket_deployment_variables`	Variable scoped to a deployment environment
`bitbucket_pipeline_variables`	Repository-scoped variable (formerly `bitbucket_repository_variable` in DrFaust92)
`bitbucket_workspace_pipeline_variables`	Workspace-scoped variable shared across repos
`bitbucket_commit_file`	Commits a file (including `bitbucket-pipelines.yml`) — sparse typed schema, use `request_body` for content / branch / message
`bitbucket_pipeline_schedules`	Scheduled pipeline runs

Combined Workspace — Bitbucket and Dynatrace in One Apply

The load-bearing pattern: a single Terraform workspace provisions the Bitbucket repo, enables pipelines on it, creates deployment environments, sets the secured variables holding Dynatrace credentials, commits the `bitbucket-pipelines.yml` file, **and** stands up the Dynatrace configuration the pipeline will deploy. One `terraform apply` brings up the whole CI/CD-to-Dynatrace path.

Why this is useful: for net-new tenants or new app teams, you can codify the entire onboarding (repo, pipeline, Dynatrace tenant config) in a single artifact. The drift-detection story is also unified — one `terraform plan` shows drift in both Bitbucket and Dynatrace.

> **Verify against current provider docs at use time.** The FabianSchurig provider is pre-1.0; some `request_body` shapes below (especially for `bitbucket_commit_file`) are best-effort against the May 2026 documentation. Confirm against the registry docs before applying.

main.tf:

```
terraform {
  required_providers {
    bitbucket = {
      source  = "FabianSchurig/bitbucket"
      version = "~&gt; 0.15"  # Pre-1.0; pin tightly until stable 1.x lands
    }
    dynatrace = {
      source  = "dynatrace-oss/dynatrace"
      version = "~&gt; 1.88"
    }
  }
}

# Bitbucket provider — API token auth (App Passwords are being retired by
Atlassian)
provider "bitbucket" {
  # token read from BITBUCKET_TOKEN env var; alternatively set inline:
  # token = var.bitbucket_token
}

# Dynatrace provider — combined auth (Platform Token + API Token)
provider "dynatrace" {
  dt_env_url   = var.dt_env_url
  dt_api_token = var.dt_api_token
  # Platform Token is read from DT_PLATFORM_TOKEN env var by the provider
}

variable "workspace" { type = string }
variable "dt_env_url" { type = string }
variable "dt_api_token" { type = string, sensitive = true }
variable "dt_platform_token" { type = string, sensitive = true }

# --- 1. Bitbucket repo ---
resource "bitbucket_repos" "config_repo" {
  workspace = var.workspace
  repo_slug = "dynatrace-config"

  # Less-common settings go through request_body in the generic-resource
architecture
  request_body = jsonencode({
    is_private  = true
    fork_policy = "no_forks"
    description = "Monaco / Terraform configs for Dynatrace tenant"
  })
}

# --- 2. Enable Bitbucket Pipelines on the repo ---
resource "bitbucket_pipeline_config" "config_repo_pipelines" {
  workspace = bitbucket_repos.config_repo.workspace
  repo_slug = bitbucket_repos.config_repo.repo_slug
  enabled   = "true"  # String, not bool — the generic schema preserves API
```

```
representations
}

# --- 3. Deployment environments ---
resource "bitbucket_deployments" "staging" {
  workspace = var.workspace
  repo_slug = bitbucket_repos.config_repo.repo_slug
  name      = "staging"
}

resource "bitbucket_deployments" "production" {
  workspace = var.workspace
  repo_slug = bitbucket_repos.config_repo.repo_slug
  name      = "production"
}

# --- 4. Secured variables holding Dynatrace credentials ---
# Repository-scoped (visible to every step):
resource "bitbucket_pipeline_variables" "dt_env_url" {
  workspace = var.workspace
  repo_slug = bitbucket_repos.config_repo.repo_slug
  key       = "DYNATRACE_ENV_URL"
  value     = var.dt_env_url
  secured   = "false"  # String
}

# Per-environment (different token per env — locked to that deployment):
resource "bitbucket_deployment_variables" "staging_platform_token" {
  workspace        = var.workspace
  repo_slug        = bitbucket_repos.config_repo.repo_slug
  environment_uuid = bitbucket_deployments.staging.uuid
  key              = "DYNATRACE_PLATFORM_TOKEN"
  value            = var.dt_platform_token  # In real use, source per-env tokens
separately
  secured          = "true"
}

resource "bitbucket_deployment_variables" "staging_api_token" {
  workspace        = var.workspace
  repo_slug        = bitbucket_repos.config_repo.repo_slug
  environment_uuid = bitbucket_deployments.staging.uuid
  key              = "DYNATRACE_API_TOKEN"
  value            = var.dt_api_token
  secured          = "true"
}

resource "bitbucket_deployment_variables" "prod_platform_token" {
  workspace        = var.workspace
  repo_slug        = bitbucket_repos.config_repo.repo_slug
  environment_uuid = bitbucket_deployments.production.uuid
  key              = "DYNATRACE_PLATFORM_TOKEN"
```

```
  value            = var.dt_platform_token
  secured          = "true"
}

resource "bitbucket_deployment_variables" "prod_api_token" {
  workspace        = var.workspace
  repo_slug        = bitbucket_repos.config_repo.repo_slug
  environment_uuid = bitbucket_deployments.production.uuid
  key              = "DYNATRACE_API_TOKEN"
  value            = var.dt_api_token
  secured          = "true"
}

# --- 5. Commit the bitbucket-pipelines.yml into the repo ---
# bitbucket_commit_file has a sparse typed schema; pass the commit details via
# request_body. Confirm the exact field names against the provider docs at use
# time — the pre-1.0 schema may evolve.
resource "bitbucket_commit_file" "pipelines_yml" {
  workspace = var.workspace
  repo_slug = bitbucket_repos.config_repo.repo_slug

  request_body = jsonencode({
    branch   = "main"
    message  = "ci: provision pipeline config via Terraform"
    author   = "Terraform &lt;terraform@example.com&gt;"
    files = {
      "bitbucket-pipelines.yml" = file("${path.module}/bitbucket-pipelines.yml")
    }
  })

  depends_on = [
    bitbucket_pipeline_config.config_repo_pipelines,
    bitbucket_deployment_variables.staging_platform_token,
    bitbucket_deployment_variables.staging_api_token,
    bitbucket_deployment_variables.prod_platform_token,
    bitbucket_deployment_variables.prod_api_token,
  ]
}

# --- 6. Dynatrace resources the pipeline will deploy on top of ---
# (Bootstrap resources — things you want present before the pipeline runs for
# the first time. Day-to-day app-team configs land in the repo and deploy via
# the pipeline itself.)
resource "dynatrace_management_zone_v2" "platform_baseline" {
  name = "platform-baseline"

  rules {
    type             = "ME"
    enabled          = true
    propagation_type = "HOST_TO_PROCESS_GROUP_INSTANCE"
```

```
    conditions {
      key {
        attribute = "HOST_GROUP_NAME"
      }
      string_conditions {
        operator         = "EQUALS"
        value            = "platform-baseline"
        case_sensitive   = false
      }
    }
  }
}
```

Ordering is load-bearing:

- `bitbucket_repos` must exist before `bitbucket_pipeline_config`, `bitbucket_pipeline_variables`, or `bitbucket_deployments` referencing it.
- `bitbucket_deployments` must exist before any `bitbucket_deployment_variables` referencing it (the `environment_uuid` is the API-returned identifier).
- `bitbucket_commit_file` for `bitbucket-pipelines.yml` should `depends_on` both the variables AND the `bitbucket_pipeline_config` — if the pipeline file lands in the repo before pipelines are enabled and variables are in place, the first push triggers a pipeline run that fails (either because pipelines aren't active, or because authentication has nothing to read).

The explicit `depends_on` in `bitbucket_commit_file` is the cleanest way to enforce this; Terraform's automatic dependency graph only catches references, not "must exist when this file is read."

Caveat on per-env tokens: the example above uses a single `var.dt_platform_token` for both staging and production deployment variables. In a real setup, you'd want **separate Dynatrace tokens per environment** — typically by running the staging and production applies in separate workspaces, or by sourcing per-environment tokens from a secrets manager (Vault, AWS Secrets Manager, Bitbucket OIDC-to-Vault flow).

Variables and Secrets

Bitbucket Pipelines has three variable scopes. Pick the right scope per credential:

Scope	Terraform resource	When to use
Workspace	`bitbucket_workspace_pipeline_variables`	Credentials shared across many repos in the workspace (e.g., a workspace-wide Dynatrace token if you don't separate by app).

Scope	Terraform resource	When to use
Repository	`bitbucket_pipeline_variables`	Per-repo credentials — typical for a `dynatrace-config` repo with its own Dynatrace tokens.
Deployment environment	`bitbucket_deployment_variables`	Per-environment credentials (staging vs. production tokens). The recommended scope when you want different Dynatrace tokens (or different tenants) per environment.

Variable	Scope	`secured`	Why
`DYNATRACE_ENV_URL`	Repository	No	Tenant URL is not secret; visible in every step
`DYNATRACE_PLATFORM_TOKEN`	Deployment	Yes	Per-env; write-only after creation
`DYNATRACE_API_TOKEN`	Deployment	Yes	Per-env; covers synthetics/SLOs not yet on Platform Token
`DYNATRACE_HTTP_OAUTH_PREFERENCE`	Repository	No	Static flag, not a credential

`secured = "true"` makes the variable value write-only — once set, the Bitbucket API won't return it (and Terraform won't drift-detect on the value). If you need to rotate, change the Terraform value and re-apply.

> **Note on YAML templating and secured variables.** Bitbucket Pipelines added a `${{ }}` YAML-templating mechanism that injects workspace, repository, and custom pipeline variables into the pipeline YAML at execution time — but **secured variables are deliberately excluded** from this templating. This is a security guarantee: a Dynatrace token stored in a secured variable cannot be inadvertently rendered into a pipeline YAML field where it might leak into logs. Read your secured variables only via standard environment-variable substitution inside `script:` blocks. Two related additions worth knowing: **Shared Pipeline Variables** (steps export variables to subsequent steps in the same pipeline; 50-variable / 100KB caps) and **Input Variables for Child Pipelines** (parent → child, up to 20 variables) — both work with non-secured variables.

> **Sources:** Variables and secrets (Atlassian) — three variable scopes (workspace / repository / deployment) and precedence; recent additions (shared pipeline variables, input variables for child pipelines, YAML templating with secured-variable exclusion).

OIDC Note

Bitbucket Pipelines supports OIDC for federating to AWS, GCP, and Vault — see the Atlassian docs on Integrate Pipelines with resource servers using OIDC. The pattern is to add `oidc: true` at the pipeline step level and configure the resource server (AWS / GCP / Vault) to trust the Bitbucket OIDC issuer.

Dynatrace does not currently document a direct OIDC trust path for Bitbucket Pipelines — there's no published mechanism to issue a short-lived Dynatrace Platform Token from a Bitbucket OIDC assertion. The practical pattern remains *secured variable holding a long-lived token*, rotated on a cadence.

If you want short-lived credentials anyway, the indirection is: Bitbucket OIDC → Vault (or AWS Secrets Manager) → fetches a Dynatrace token at pipeline start. The pipeline still ends up with a token in memory, but the token in storage is in Vault, not in Bitbucket variables.

> **Sources:**
> - Bitbucket provider (FabianSchurig) — actively-maintained successor to `DrFaust92/bitbucket`; v0.15.7 as of May 2026 (pre-1.0, schema flux possible).
> - Migration guide DrFaust92 → FabianSchurig — resource-name mapping, auth changes, path-parameter renames.
> - DrFaust92/terraform-provider-bitbucket — long-standing provider, now in maintenance mode (issue #242, March 2026).
> - Get started with Bitbucket Pipelines (Atlassian) — Pipelines overview.
> - API tokens (Atlassian) and App passwords (Atlassian) — auth-token landscape; App Passwords are being retired in favor of API tokens.
> - Integrate Pipelines with resource servers using OIDC (Atlassian) — Bitbucket OIDC for AWS/GCP/Vault.

---

6. ArgoCD Integration

For Kubernetes-native GitOps, use ArgoCD with Dynatrace configs.

ArgoCD Application for Monaco Configs

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: dynatrace-config
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/org/dynatrace-config.git
    targetRevision: HEAD
    path: projects
  destination:
    server: https://kubernetes.default.svc
    namespace: dynatrace
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
```

ArgoCD for Dynatrace Operator (DynaKube)

Deploy and manage the Dynatrace Operator via ArgoCD:

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: dynatrace-operator
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/org/k8s-platform.git
    targetRevision: HEAD
    path: dynatrace
  destination:
    server: https://kubernetes.default.svc
    namespace: dynatrace
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
    syncOptions:
      - CreateNamespace=true
```

Repository structure for Dynatrace Operator:

```
dynatrace/
├── kustomization.yaml
├── namespace.yaml
├── operator.yaml          # Dynatrace Operator deployment
├── dynakube.yaml          # DynaKube custom resource
└── secrets/
    └── dynakube-secret.yaml  # External Secrets reference
```

External Secrets for Token Management

Use External Secrets Operator to sync tokens from secret stores:

```
apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata:
  name: dynakube-tokens
  namespace: dynatrace
spec:
  refreshInterval: 1h
  secretStoreRef:
    name: aws-secrets-manager
    kind: ClusterSecretStore
  target:
    name: dynakube
    creationPolicy: Owner
  data:
    - secretKey: apiToken
      remoteRef:
        key: dynatrace/api-token
    - secretKey: dataIngestToken
      remoteRef:
        key: dynatrace/data-ingest-token
```

Using Config Management Plugins

argocd-cm ConfigMap:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: argocd-cm
  namespace: argocd
data:
  configManagementPlugins: |
    - name: monaco
      generate:
        command: ["sh", "-c"]
        args: ["monaco deploy manifest.yaml --environment $ARGOCD_ENV_ENVIRONMENT"]
```

---

7. FluxCD Integration

FluxCD provides an alternative GitOps approach with a pull-based reconciliation model.

FluxCD vs ArgoCD

Feature	FluxCD	ArgoCD
Architecture	Controller-based	Server + UI
UI	Minimal (Weave GitOps)	Built-in web UI
Multi-tenancy	Namespace-based	Project-based
Helm Support	Native HelmRelease	Application CRD
Image Automation	Built-in	Requires Argo CD Image Updater

FluxCD for Dynatrace Operator

GitRepository source:

```yaml
apiVersion: source.toolkit.fluxcd.io/v1
kind: GitRepository
metadata:
  name: dynatrace-config
  namespace: flux-system
spec:
  interval: 1m
  url: https://github.com/org/k8s-platform
  ref:
    branch: main
  secretRef:
    name: github-token
```

Kustomization for Dynatrace:

```yaml
apiVersion: kustomize.toolkit.fluxcd.io/v1
kind: Kustomization
metadata:
  name: dynatrace
  namespace: flux-system
spec:
  interval: 10m
  targetNamespace: dynatrace
  sourceRef:
    kind: GitRepository
    name: dynatrace-config
  path: ./dynatrace
  prune: true
  healthChecks:
    - apiVersion: apps/v1
      kind: Deployment
      name: dynatrace-operator
      namespace: dynatrace
```

FluxCD HelmRelease for Dynatrace Operator

Deploy via Helm with FluxCD:

```
apiVersion: source.toolkit.fluxcd.io/v1beta2
kind: HelmRepository
metadata:
  name: dynatrace
  namespace: flux-system
spec:
  interval: 1h
  url: https://raw.githubusercontent.com/Dynatrace/dynatrace-
operator/main/config/helm/repos/stable

---
apiVersion: helm.toolkit.fluxcd.io/v2beta1
kind: HelmRelease
metadata:
  name: dynatrace-operator
  namespace: dynatrace
spec:
  interval: 5m
  chart:
    spec:
      chart: dynatrace-operator
      version: "&gt;=1.0.0"
      sourceRef:
        kind: HelmRepository
        name: dynatrace
        namespace: flux-system
  values:
    installCRD: true
```

SOPS for Secret Management with FluxCD

Use SOPS to encrypt secrets in Git:

```yaml
apiVersion: kustomize.toolkit.fluxcd.io/v1
kind: Kustomization
metadata:
  name: dynatrace-secrets
  namespace: flux-system
spec:
  interval: 10m
  sourceRef:
    kind: GitRepository
    name: dynatrace-config
  path: ./dynatrace/secrets
  prune: true
  decryption:
    provider: sops
    secretRef:
      name: sops-age
```

---

8. Dynatrace Operator GitOps Patterns

When deploying the Dynatrace Operator via GitOps, follow these patterns for production environments.

> **Important:** Use `apiVersion: dynatrace.com/v1beta5` or `v1beta6` for Dynatrace Operator 1.8.x. Earlier versions (v1beta1, v1beta2) are deprecated and no longer supported.

Multi-Cluster Deployment Pattern

For organizations with multiple Kubernetes clusters:

```
platform-gitops/
├── clusters/
│   ├── production-east/
│   │   └── dynatrace/
│   │       ├── kustomization.yaml
│   │       └── dynakube-patch.yaml     # Cluster-specific overrides
│   ├── production-west/
│   │   └── dynatrace/
│   │       ├── kustomization.yaml
│   │       └── dynakube-patch.yaml
│   └── staging/
│       └── dynatrace/
│           ├── kustomization.yaml
│           └── dynakube-patch.yaml
└── base/
    └── dynatrace/
        ├── kustomization.yaml
        ├── namespace.yaml
        ├── operator.yaml
        └── dynakube.yaml               # Base DynaKube config
```

Base kustomization.yaml:

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
  - namespace.yaml
  - https://github.com/Dynatrace/dynatrace-
operator/releases/latest/download/kubernetes.yaml
  - dynakube.yaml
```

Cluster overlay (production-east):

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
  - ../../base/dynatrace
patchesStrategicMerge:
  - dynakube-patch.yaml
```

Cluster-specific patch:

```
apiVersion: dynatrace.com/v1beta5
kind: DynaKube
metadata:
  name: dynakube
  namespace: dynatrace
spec:
  oneAgent:
    cloudNativeFullStack:
      args:
        - --set-host-group=production-east
```

Multi-Tenant Observability Pattern

For clusters serving multiple teams with different Dynatrace tenants:

```
# Team A - uses tenant-a.live.dynatrace.com
apiVersion: dynatrace.com/v1beta5
kind: DynaKube
metadata:
  name: team-a-dynakube
  namespace: dynatrace
spec:
  apiUrl: https://tenant-a.live.dynatrace.com/api
  tokens: team-a-tokens
  oneAgent:
    cloudNativeFullStack:
      namespaceSelector:
        matchLabels:
          dynatrace-tenant: team-a

---
# Team B - uses tenant-b.live.dynatrace.com
apiVersion: dynatrace.com/v1beta5
kind: DynaKube
metadata:
  name: team-b-dynakube
  namespace: dynatrace
spec:
  apiUrl: https://tenant-b.live.dynatrace.com/api
  tokens: team-b-tokens
  oneAgent:
    cloudNativeFullStack:
      namespaceSelector:
        matchLabels:
          dynatrace-tenant: team-b
```

GitOps Health Checks

Configure sync health checks to verify Dynatrace deployment:

ArgoCD health check:

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: dynatrace
spec:
  # ... source config ...
  ignoreDifferences:
    - group: dynatrace.com
      kind: DynaKube
      jsonPointers:
        - /status  # Ignore status field changes
```

FluxCD health check:

```
apiVersion: kustomize.toolkit.fluxcd.io/v1
kind: Kustomization
metadata:
  name: dynatrace
spec:
  healthChecks:
    - apiVersion: apps/v1
      kind: Deployment
      name: dynatrace-operator
      namespace: dynatrace
    - apiVersion: apps/v1
      kind: DaemonSet
      name: dynakube-oneagent
      namespace: dynatrace
```

Sealed Secrets Alternative

For Kubernetes-native secret encryption:

```
apiVersion: bitnami.com/v1alpha1
kind: SealedSecret
metadata:
  name: dynakube
  namespace: dynatrace
spec:
  encryptedData:
    apiToken: AgBy8BYo...encrypted...
    dataIngestToken: AgCtr4s2...encrypted...
```

Generate sealed secrets:

```
# Seal the secret (use your actual tokens from Dynatrace)
kubectl create secret generic dynakube \
  --from-literal=apiToken=&lt;your-api-token&gt; \
  --from-literal=dataIngestToken=&lt;your-data-ingest-token&gt; \
  --dry-run=client -o yaml | \
  kubeseal --format yaml &gt; sealed-dynakube.yaml
```

---

9. Best Practices

Security

Practice	Description
Masked secrets	Always mask API tokens in CI/CD
Protected branches	Require reviews for main branch
Environment protection	Manual approval for production
Token rotation	Rotate tokens regularly
Least privilege	Minimal token scopes
External Secrets	Use ESO, SOPS, or Sealed Secrets for K8s
Vault for runtime creds	Retrieve tokens at pipeline runtime instead of static CI/CD secrets
Combined auth	Use Platform Token + API Token for full resource coverage (v1.88.0)
Policy-as-code	OPA/Conftest or Sentinel to enforce resource allowlists and mandatory tagging

Practice	Description
State file access	Lock down HCP Terraform workspace permissions — state files may contain OAuth credentials
Two-pipeline model	Separate IAM management (central team) from config management (LOB teams)
Single SA writer	Only the pipeline SA writes to production; all humans get read-only access

Workflow Design

Practice	Description
Validate first	Always validate before deploy
Dry run PRs	Show what would change
Plan as PR comment	Post `terraform plan` output directly on PRs for reviewer context
Staged rollout	Dev → Staging → Production
Manual gates	Require approval for production
Rollback plan	Know how to revert changes
Health checks	Verify deployment success
Drift detection	Schedule `terraform plan -detailed-exitcode` to catch ClickOps
Reusable workflows	`workflow_call` for consistent deployment across repos

GitOps Tool Selection

Use Case	Recommended Tool
Simple CI/CD	GitHub Actions / GitLab CI
K8s-native, UI preferred	ArgoCD
K8s-native, Helm-first	FluxCD
Multi-cluster enterprise	ArgoCD or FluxCD

Pull Request Template

.github/PULL_REQUEST_TEMPLATE.md:

```
&lt;a id="dynatrace-configuration-change"&gt;&lt;/a&gt;
## Dynatrace Configuration Change
### Description
&lt;!-- What configuration is being changed? --&gt;

### Type of Change
- [ ] New configuration
- [ ] Modification to existing config
- [ ] Deletion of config

### Environments Affected
- [ ] Development
- [ ] Staging
- [ ] Production

### Validation
- [ ] Monaco validate passed
- [ ] Dry run successful
- [ ] Tested in dev environment

### Rollback Plan
&lt;!-- How to revert if issues arise? --&gt;
```

Notification Integration

Add deployment notifications:

```
# GitHub Actions notification step
- name: Notify Slack
  if: always()
  uses: slackapi/slack-github-action@v1
  with:
    payload: |
      {
        "text": "Dynatrace config deployment: ${{ job.status }}",
        "blocks": [
          {
            "type": "section",
            "text": {
              "type": "mrkdwn",
              "text": "*Deployment to ${{ github.ref_name }}*\nStatus: ${{
job.status }}\nCommit: ${{ github.sha }}"
            }
          }
        ]
      }
  env:
    SLACK_WEBHOOK_URL: ${{ secrets.SLACK_WEBHOOK }}
```

---

10. Governance Architecture

The previous sections covered individual governance tools (Vault, OPA, Sentinel, drift detection). This section synthesizes them into a cohesive **enterprise governance architecture** for Dynatrace configuration management.

The Two-Pipeline Model

Enterprise Dynatrace governance requires **two distinct pipelines** with separate identities, scopes, and cadences:

Pipeline A — IAM Pipeline ("Who can do what")

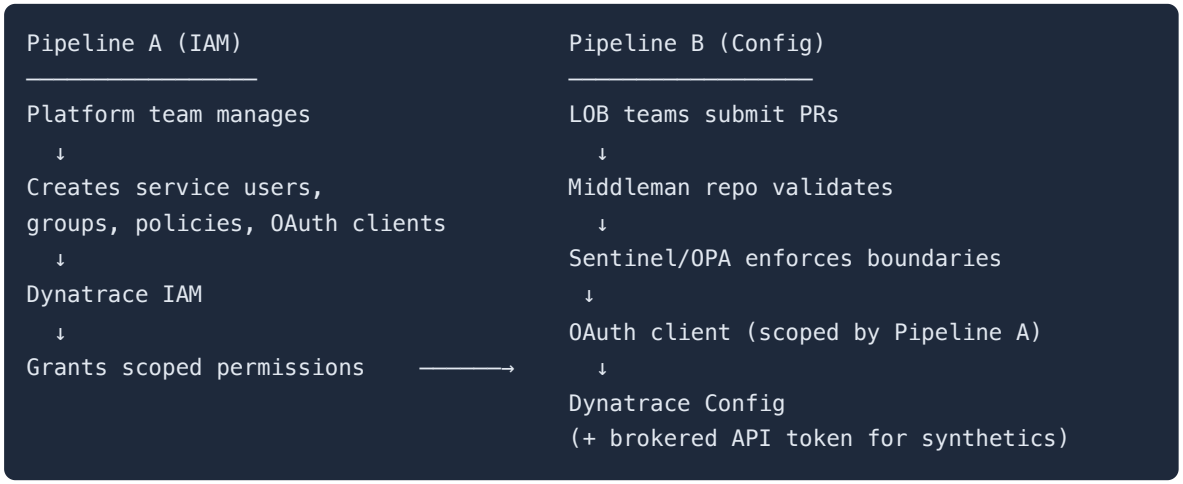
Aspect	Detail
Owner	Central Platform / Observability team
Identity	Service User + OAuth client scoped to IAM resources
Manages	Groups, policies, policy bindings, environment-level permissions (`dynatrace_iam_*`)

Aspect	Detail
Governance	Sentinel/OPA prevents privilege escalation (no `account:admin`, no cross-team bindings)
Cadence	Infrequent — runs when teams onboard, roles change, or environments are provisioned
Repo	`dt-global-config` or equivalent central IAM repo

Pipeline B — Configuration Pipeline ("The actual work")

Aspect	Detail
Owner	Per-team / per-LOB (via middleman repo or brokered self-service)
Identity	Service User + OAuth client scoped to the config domains Pipeline A granted
Manages	Workflows, dashboards, segments, alerting profiles, auto-tags, management zones
Auth	Dual-auth (OAuth + API token) when synthetics are in scope
Governance	Sentinel/OPA enforces naming, tagging, MZ boundaries, allowed resource types
Cadence	Frequent — runs on every config change promotion through environments
Repo	`dt-lob-<lobname>` or team-specific config repos

Pipeline B **can only do what Pipeline A has granted**. Its OAuth client's effective permissions are bounded by the IAM policies, group memberships, and environment scoping that Pipeline A defined.



> **Key principle:** Pipeline A creates the identities and permissions. Pipeline B operates under those grants. Separation of concerns between access governance and

> configuration management.

Operational Model: Single SA Writer

In practice, the two-pipeline architecture produces a clear operational rule: **all human users have read-only access in production, and only one service account can write.**

Teams build and validate configuration in a **dev/test tenant** where they have UI write access. Once tested, the provisioning pipeline — running as the single SA (service account) — promotes that configuration to production. No human ClickOps in prod, and a single auditable write path.

```
Dev Tenant                              Prod Tenant
──────────                              ───────────
Humans: read + write (UI)               Humans: read-only
  ↓                                       ↑
Teams build &amp;amp; test config                 Pipeline promotes tested config
  ↓                                       ↑
Pipeline B validates    ──────────────→ SA user writes (only writer)
```

This model works with or without Sentinel. It directly addresses the access scoping challenge: within a single tenant, you cannot easily separate write access between teams for v1 resources. By splitting dev (where humans create) from prod (where only the pipeline writes), you get isolation through environment boundaries rather than token scoping. The SA's credentials live in Vault or HCP Terraform — never in human hands.

Defense-in-Depth Governance Layers

No single tool provides complete governance. Enterprise Dynatrace configuration management uses **layered controls** where each layer compensates for the limitations of the others:

Layer	Role	Tool / Mechanism
Dynatrace IAM	Source of truth for access control	IAM policies, groups, bindings (see **AUTOM-04**)
Service User + OAuth	Scoped pipeline identity for Gen3 resources	OAuth client credentials
Dual Auth (OAuth + API)	Full coverage when pipeline manages both Gen3 and v1 resources	Combined provider config

Layer	Role	Tool / Mechanism
Sentinel / OPA	Governance guardrails on what Terraform can do	Policy-as-code (see above)
Vault	Runtime credential management with expiration and audit	HashiCorp Vault (see above)
PR Review + CODEOWNERS	Human approval gate	GitHub/GitLab branch protection
Module Allowlists	Only approved Terraform modules in use	Sentinel module restrictions or repo-level controls
State File Access Control	Prevent credential extraction from Terraform state	HCP Terraform workspace permissions

> **State file risk:** If a team can read the state of a middleman workspace in HCP Terraform, they could potentially extract the OAuth client credentials or see IAM bindings they should not access. Lock down workspace-level permissions in HCP Terraform appropriately.

What Sentinel / OPA Can and Cannot Do

Sentinel / OPA CAN	Sentinel / OPA CANNOT
Restrict which Terraform modules and resource types may be used	Integrate with Dynatrace IAM at runtime
Enforce naming conventions, tagging, and MZ boundaries	Reduce the runtime permissions of a Dynatrace token
Prevent privilege escalation in IAM-as-code	Replace Dynatrace RBAC / IAM policies
Validate team-to-object ownership in brokered self-service	Run outside HCP Terraform (Sentinel only — use OPA/Conftest for CI)

> Sentinel does not *grant* access. Sentinel decides whether Terraform is *allowed to grant* access. Dynatrace IAM remains the source of truth for what the pipeline identity can actually do.

Framing for Security Reviewers

When presenting this architecture to security teams or auditors, document the v1 Synthetic limitation as an **accepted platform constraint with compensating controls**:

1. **Gen3/platform resources** — genuinely scoped via Service User + OAuth + IAM policies
2. **v1 Synthetic monitors** — brokered API token access through central pipeline (teams never hold the token directly)
3. **Pipeline guardrails** — Sentinel/OPA enforce naming, tagging, MZ boundaries, and resource type restrictions
4. **Credential management** — Vault provides runtime retrieval, automatic expiration, and access audit logging
5. **Audit trail** — Git history + PR reviews + Dynatrace audit logs provide full change traceability

This framing is much stronger than trying to pretend the v1 API scoping gap does not exist. It demonstrates a mature, defense-in-depth approach to a known platform limitation.

---

11. Next Steps

Deployment Event Tracking

Send deployment events to Dynatrace:

```
- name: Send Deployment Event
  run: |
    curl -X POST "${{ secrets.DT_URL }}/api/v2/events/ingest" \
      -H "Authorization: Api-Token ${{ secrets.DT_TOKEN }}" \
      -H "Content-Type: application/json" \
      -d '{
        "eventType": "CUSTOM_DEPLOYMENT",
        "title": "Dynatrace Config Deployment",
        "properties": {
          "source": "github-actions",
          "commit": "${{ github.sha }}",
          "branch": "${{ github.ref_name }}"
        }
      }'
```

Continue the Series

Next Notebook	Focus
AUTOM-08: Migration Automation	Bulk configuration transfer between tenants

Additional Resources

- GitHub Actions Documentation
- GitLab CI/CD Documentation
- ArgoCD Documentation
- FluxCD Documentation
- Dynatrace Operator
- Monaco GitHub Repository
- External Secrets Operator
- OPA/Conftest
- Sentinel Documentation
- HashiCorp Vault Action
- Dynatrace OAuth Client Guide

Summary

In this notebook, you learned:

- GitOps fundamentals for Dynatrace configuration
- Setting up GitHub Actions and GitLab CI/CD pipelines
- **Combined auth** (Platform Token + API Token) for full resource coverage in Terraform pipelines
- **Vault integration** for runtime credential retrieval with short-lived tokens
- **Policy-as-code gates** using OPA/Conftest and Sentinel for governance enforcement
- **Sentinel limitations** — it governs Terraform plans, not Dynatrace API permissions at runtime
- **Drift detection** workflows to catch manual UI changes
- **Reusable workflows** (`workflow_call`) for multi-team organizations
- ArgoCD integration with External Secrets for token management
- FluxCD with HelmRelease and SOPS for secret encryption

- Dynatrace Operator GitOps patterns for multi-cluster and multi-tenant environments
- **Two-pipeline model**: Pipeline A (IAM) creates identities, Pipeline B (Config) applies configuration under those grants
- **Single SA writer**: only the pipeline SA writes to production; humans develop in dev/test
- **Defense-in-depth**: layered governance from Dynatrace IAM through Sentinel/OPA, Vault, PR gates, and state file controls

> **Key Takeaway:** CI/CD integration transforms Dynatrace config management into a software development workflow. Use a two-pipeline model to separate IAM governance from configuration management. Enforce single-SA-writer in production. Layer Vault, OPA/Sentinel, and Dynatrace IAM policies for defense-in-depth — no single tool provides complete governance.

Continue to AUTOM-08: Migration Automation to learn bulk configuration transfer.

Community Resources & CI/CD Examples

The following GitHub repositories provide working CI/CD pipelines, validation scripts, and platform engineering references:

Pipeline Observability Samples (in `dynatrace-configuration-as-code-samples`)

Both Monaco and Terraform variants are available for each CI/CD platform:

CI/CD Platform	Monaco Sample	Terraform Sample
GitHub Actions	`github_pipeline_observability`	`github_pipeline_observability_terraform`
GitLab CI	`gitlab_pipeline_observability`	`gitlab_pipeline_observability_terraform`
Azure DevOps	`azure_devops_observability`	--
ArgoCD	`argocd_observability`	`argocd_observability_terraform`

All samples ingest SDLC events via OpenPipeline and include pre-built dashboards.

CI Validation Scripts (in `dynatrace-configuration-as-code-samples/scripts/ci/`)

Script	Purpose
`validate-monaco.sh`	Validate Monaco configuration in CI
`validate-terraform.sh`	Validate Terraform configuration in CI
`check-secrets.sh`	Detect hardcoded secrets before commit

CI/CD Automation Tools

Repository	Description
dynatrace-automation-tools	CLI for Site Reliability Guardian automation and deployment event ingestion in CI/CD (v1.0.3)
monaco-demo	Working GitHub Actions workflow for Monaco deploy with `.github/workflows/monaco.yml`

Platform Engineering References

Repository	Description
platform-engineering-demo	Full IDP reference: ArgoCD + Backstage + Keptn + OpenTelemetry + Dynatrace
demo-crossplane	Crossplane + Terraform provider for continuous GitOps reconciliation (no manual pipeline triggers)
obslab-composite-srg	Composite Site Reliability Guardians with GitHub Actions
obslab-release-validation	Release validation using k6, business events, workflows, and SRG
